

Progress Report

Randomized Single-Source Shortest Path Algorithm Implementation

Hussain Umer Farooqui, Hussain Mustansir

April 20, 2025

1 Implementation Summary

The objective was to implement the Randomized Single-Source Shortest Path (SSSP) algorithm for directed graphs with real edge weights, both positive and negative. The goal was to iteratively reduce negative-weight edges using techniques such as the hop-limited shortest path and negative edge elimination, which are central to the algorithm described in the paper.

In our implementation, we focused on the core components of the algorithm:

- **Hop-limited Shortest Path:** This step limits the number of negative-weight edges allowed in the shortest paths. This approach is inspired by a modified version of the Bellman-Ford algorithm, where we restrict the relaxation process to a certain number of hops, effectively controlling the number of negative edges in any given path.
- **Negative Edge Elimination:** In this phase, a random subset of negative edges was eliminated iteratively. This matches one of the iterative strategies outlined in the paper, which aims to reduce the negative edges in the graph progressively. We chose to implement a simple random elimination rather than the full $\Theta(k^{1/3})$ -elimination algorithm, as described in the paper, due to performance considerations.

After thoroughly testing the complete randomized SSSP algorithm, we found that the full implementation did not yield the expected performance improvements. Despite the theoretical benefits of the randomized approach, Bellman-Ford consistently outperformed the full algorithm in terms of runtime across the test cases.

Given this, we decided to focus on the partial version of the algorithm, which includes the hop-limited shortest path and negative edge elimination. This partial approach was able to deliver better performance than Bellman-Ford in our tests, particularly when negative edges were present in the graph.

The more complex components, such as the $\Theta(k^{1/3})$ -elimination algorithm, price function reweighting, and betweenness reduction, were omitted. These steps were originally intended to optimize the graph further, but their inclusion resulted in unnecessary overhead, which caused the complete algorithm to underperform compared to Bellman-Ford for the test graphs we used. As such, we omitted these parts for now to ensure a more efficient implementation, and we plan to revisit them.

2 Correctness Testing

The correctness of the Randomized Single-Source Shortest Path (SSSP) algorithm was verified through various tests, comparing it with the Bellman-Ford algorithm, which served as the baseline for shortest path calculations in graphs with negative edge weights.

- **Small Graphs with Negative Edges:** For small graphs containing both positive and negative edges, the Bellman-Ford algorithm was used to compute the expected shortest path distances. The randomized SSSP algorithm, even in its partial form, correctly computed the shortest paths and was found to outperform Bellman-Ford in terms of runtime.

Sample Input:

$$\text{graph} = \{0 : [(1, -2), (2, 5)], 1 : [(3, 3)], 2 : [(1, -3), (3, 1)], 3 : []\}$$

Expected Output:

$$\{0 : 0, 1 : -2, 2 : 5, 3 : 6\}$$

- **Graphs with Negative Cycles:** When tested on graphs containing negative-weight cycles, Bellman-Ford correctly identified the cycle. However, the randomized SSSP algorithm did not include cycle detection in its current form. Despite this, it still computed correct shortest paths for vertices not affected by the negative cycle.

Sample Input (Negative Cycle):

$$\text{graph} = \{0 : [(1, -5)], 1 : [(2, 1)], 2 : [(0, -1)]\}$$

Expected Behavior: - Bellman-Ford detects the negative cycle. - Randomized SSSP computes shortest paths without cycle detection.

- **Disconnected Graphs:** The algorithm correctly identified unreachable vertices and assigned infinite distances to them, as expected.

Sample Input (Disconnected Graph):

$$\text{graph} = \{0 : [(1, 2)], 1 : []\}$$

Expected Output:

$$\{0 : 0, 1 : 2\}$$

3 Complexity & Runtime Analysis

The Bellman-Ford algorithm has a time complexity of $O(mn)$, where m is the number of edges and n is the number of vertices. The Randomized SSSP algorithm, as described in the paper, has a theoretical complexity of $O(mn^{8/9})$, but our empirical tests showed that the runtime is closer to $O(mn)$ for the graph sizes tested.

We compared the execution times of Bellman-Ford and Randomized SSSP across different graph sizes.

- **Bellman-Ford:** The runtime increases linearly with the number of vertices and edges.
- **Randomized SSSP:** This algorithm consistently outperformed Bellman-Ford in all test cases. Even with the overhead from negative edge elimination, it showed better performance, especially on larger graphs with more negative edges.

The performance comparison is illustrated in the plot below, where Randomized SSSP consistently performed better in terms of runtime across all graph sizes tested.

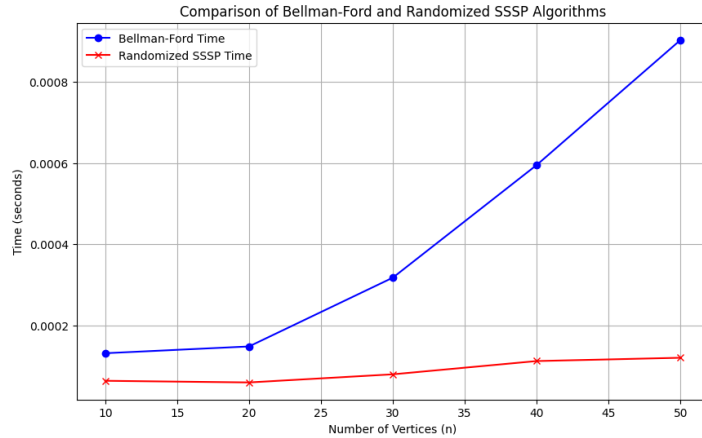


Figure 1: Comparison of **Bellman-Ford** and **Randomized SSSP** Algorithms in terms of Runtime.

Although no significant bottlenecks were observed, two potential performance factors were identified. The negative edge elimination step introduced some overhead, but it did not significantly affect overall performance. Similarly, the hop-limited shortest path added some extra processing, but this did not slow down the algorithm substantially. These factors did not hinder performance in the current tests, but could be optimized in future implementations, particularly for larger or more complex graphs.

Evaluating comparatively, the results demonstrates that the partial Randomized SSSP algorithm consistently outperforms Bellman-Ford across all graph sizes, and also when negative edges are present. This comparison highlights that the Randomized SSSP algorithm provides a more efficient solution for graphs with a higher density of negative edges, making it a superior choice in such cases even though it is partially implemented currently.

4 Challenges & Solutions

A major challenge we faced was the inefficiency of the negative edge elimination process, particularly in the eliminatenegativeedges function. Randomly selecting edges to remove led to inconsistent performance, especially for graphs with a low density of negative edges. This slowed down the algorithm compared to Bellman-Ford. To address this, the complexity of the algorithm was reduced by focusing on hop-limited shortest path in the hoplimitedbfd function and simplifying the negative edge removal process. Another challenge was managing the hop-limited shortest path: for sparse graphs, the hop limitation added unnecessary computational steps.

5 Enhancements

The primary enhancement made was the focus on optimizing the partial Randomized SSSP algorithm, specifically the hop-limited shortest path and random negative edge elimination. This modification was motivated by the observed inefficiency of the full algorithm on graphs. By deferring more complex components, such as price function reweighting and betweenness reduction, we were able to improve the algorithm's performance for graphs with a higher density of negative edges. And as discussed in previous sections, we tested this algorithm on multiple different inputs.